

☑ Final Presentation Flow

- **Member 1:** Introduction + Bisection Algorithm
- **Member 2:** Bisection Simulation
- **Member 3:** Regula Falsi Algorithm + Simulation
- **Member 4:** Python Implementation
- **Member 5:** Website Demonstration + Conclusion

This sequence follows exactly what your lecturer requested:

Algorithm → Simulation → Python Implementation → Website Demonstration → Conclusion. It should fit well into a **15–20 minute group presentation** with five speakers.

MEMBER-1 PRESENTATION SCRIPT

(Introduction + Problem Statement + Bisection Algorithm)

Assalamu Alaikum and Good Morning respected sir and my dear classmates.

Today, our group is going to present our project titled "**Numerical Root Finding Suite using Python Flask.**"

This project is a web-based application developed to solve nonlinear equations using two numerical methods: the **Bisection Method** and the **Regula Falsi Method**, also known as the False Position Method.

Before discussing our implementation, let us first understand the problem we are trying to solve.

In mathematics and engineering, many equations such as $x^3 - x - 2 = 0$ cannot be solved easily using algebraic formulas.

In such situations, we use **numerical methods** to obtain an approximate root with high accuracy.

Our web application allows users to enter any valid mathematical equation, specify an interval, choose the desired numerical method, and calculate the root automatically.

Along with the numerical result, the application also displays the iteration table, graphical visualization, step-by-step animation, and provides a downloadable PDF report.

Let us first discuss the **Bisection Method**.

The Bisection Method is one of the simplest and most reliable numerical techniques for finding the root of a continuous function.

It is based on the **Intermediate Value Theorem**, which states that if a continuous function changes its sign over an interval, then at least one root exists within that interval.

Therefore, before starting the algorithm, we must verify that **$f(a) \times f(b)$ is less than zero**.

If this condition is not satisfied, the algorithm cannot guarantee the existence of a root, and our application displays an error message to the user.

Once the interval is validated, the algorithm begins by calculating the midpoint of the interval using the formula:

$$c = (a + b) / 2$$

After calculating the midpoint, the function value at **c**, or **$f(c)$** , is evaluated.

The algorithm then compares the sign of **$f(a)$** with **$f(c)$** .

If **$f(a)$** and **$f(c)$** have opposite signs, the root must lie between **a** and **c**, so the upper bound **b** is replaced by **c**.

Otherwise, the root lies between **c** and **b**, and the lower bound **a** is replaced by **c**.

This process continues repeatedly, reducing the interval size in every iteration.

The algorithm terminates when the function value becomes sufficiently close to zero or when the interval size becomes smaller than the specified tolerance.

Because the interval is reduced in every iteration, the Bisection Method always converges to the root, although it may require more iterations than some other numerical methods.

In our application, this entire process is performed automatically after the user provides the equation, interval, tolerance, and maximum number of iterations.

Now that we have understood the algorithm of the Bisection Method, I would like to invite my teammate to demonstrate its step-by-step simulation using an actual numerical example.

Thank you.

Member 2 Presentation Script (Bisection Method Simulation)

(About 20–25 lines, 3–4 minutes)

Thank you. Now I will demonstrate how the Bisection Method works through a practical example.

Suppose we want to solve the equation $f(x) = x^3 - x - 2 = 0$.

First, we choose an initial interval $a = 1$ and $b = 2$.

Before starting the iteration, we calculate $f(1) = -2$ and $f(2) = 4$.

Since one value is negative and the other is positive, the condition $f(a) \times f(b) < 0$ is satisfied. This confirms that a root exists within this interval.

In the **first iteration**, we calculate the midpoint using the formula $c = (a + b) / 2$, which gives $c = 1.5$.

Next, we evaluate $f(1.5)$. Since the sign of $f(1.5)$ is different from $f(2)$, we update our interval to $[1.5, 2]$.

In the **second iteration**, we again calculate the midpoint of the new interval, evaluate the function, and determine which half contains the root.

This process continues in every iteration. Each time, the interval becomes smaller, bringing us closer to the actual root.

One important point is that the Bisection Method always reduces the search interval by half, which makes it a stable and reliable numerical method.

In our web application, every iteration is displayed in a table showing the values of a , b , c , and $f(c)$.

Users can clearly observe how the interval changes from one iteration to the next until the stopping condition is reached.

Our application also provides a **step-by-step animation**, making it easier to understand how the algorithm converges toward the root.

This visual representation is especially helpful for students because they can follow the complete iteration process instead of seeing only the final answer.

After all iterations are completed, the application displays the final approximated root along with a graph of the function.

This completes the simulation of the Bisection Method.

Now I would like to invite my next teammate, who will explain the Regula Falsi Method and its algorithm.

Thank you.

-  **Member 5:** Website Demonstration + Conclusion

Below is **Member 3's script**.

Member 3 Presentation Script (Regula Falsi Method & Simulation)

(About 20–25 lines, 3–4 minutes)

Thank you. Now I will explain the second numerical method used in our project, which is the Regula Falsi Method, also known as the False Position Method.

Like the Bisection Method, the Regula Falsi Method is also used to find the root of a nonlinear equation within a given interval.

It also requires that the function values at the two endpoints have opposite signs, meaning $f(a) \times f(b) < 0$.

However, the main difference lies in the way the next approximation is calculated.

Instead of taking the midpoint of the interval, the Regula Falsi Method draws a straight line between the two endpoints of the function.

The point where this straight line intersects the x-axis is taken as the next approximation of the root.

The formula used in this method is:

$$c = b - [f(b) \times (a - b)] / [f(a) - f(b)]$$

This method often reaches the root faster because it considers the function values instead of simply dividing the interval into two equal parts.

Now let us understand this with the same example, $f(x) = x^3 - x - 2$, where the initial interval is **[1, 2]**.

First, we calculate the values of **f(1)** and **f(2)**, confirming that a root exists within the interval.

Using the Regula Falsi formula, we calculate the first approximation **c** instead of the midpoint.

Then we evaluate $f(c)$ and check which subinterval contains the root.

Depending on the sign of $f(c)$, either the lower bound or the upper bound is updated.

This process continues until the function value becomes sufficiently close to zero or the required tolerance is achieved.

In our web application, every iteration is stored and displayed in a table, allowing users to observe how the approximation changes step by step.

The application also provides graphical visualization and step animation, making the convergence process much easier to understand.

Compared to the Bisection Method, the Regula Falsi Method generally converges faster because it uses linear interpolation instead of the midpoint.

This makes it a more efficient choice for many nonlinear equations.

Member 4 Presentation Script (Python Implementation)

(30–35 lines | Around 5–6 minutes)

Thank you. Now I will explain the Python implementation of our project, which is the core part of this application.

Our project is developed using the **Flask Framework**, a lightweight Python web framework that allows us to connect the user interface with the backend numerical algorithms.

In simple words, Flask acts as a bridge between the HTML pages and the Python code that performs all the calculations.

At the beginning of our program, we import several libraries, and **each library has a specific responsibility** in our application.

For example, **Flask** is responsible for handling web requests and displaying different web pages.

We use the **SymPy** library because users enter equations as plain text, such as $x^3 - x - 2$. SymPy converts this text into a mathematical expression that Python can understand and evaluate.

To achieve this, we created a function called **parse_equation()**.

Inside this function, the program first replaces the '^' **operator with '****, because Python uses double asterisks for exponentiation.

Then, using **sympify()**, the equation is converted into a symbolic mathematical expression.

Finally, **lambdify()** transforms the symbolic expression into an executable Python function, allowing us to calculate values such as **f(1)**, **f(2)**, or **f(c)** during every iteration.

Next, our application defines different **Flask Routes**.

The **Home Route** displays the main page, while separate routes are created for the **Bisection Method**, the **False Position Method**, and **PDF generation**.

When the user clicks the **Calculate Root** button, the browser sends a **POST request** to the Flask server.

The server then receives all the input values, including the equation, lower bound, upper bound, tolerance, and maximum number of iterations.

One important validation step is performed before starting the numerical algorithm.

The program checks whether **$f(a) \times f(b)$ is less than zero**.

If this condition is not satisfied, the application immediately stops the calculation and displays an error message because the selected interval cannot guarantee the existence of a root.

If the interval is valid, the selected numerical algorithm begins.

For the **Bisection Method**, the midpoint is calculated using the formula **$c = (a + b) / 2$** .

For the **False Position Method**, the next approximation is calculated using the **Regula Falsi interpolation formula**, which usually converges faster than the midpoint approach.

After calculating the new approximation, the function value **f(c)** is evaluated.

Based on its sign, the program updates either the lower bound or the upper bound and continues the iteration until the stopping condition is reached.

An important feature of our implementation is that every iteration is stored instead of only displaying the final answer.

During each iteration, the values of **a**, **b**, **c**, and **f(c)** are saved inside a Python list as a dictionary.

Later, this data is converted into **JSON format** and sent to the HTML page.

Because of this implementation, our website can display the **iteration table**, create the **step-by-step animation**, and show the progress of the algorithm visually.

After completing the iterations, our application generates the graph using the **Matplotlib library**.

The graph plots the mathematical function and marks the calculated root with a highlighted point, allowing users to verify the solution visually.

Instead of saving the graph as an image file, we convert it into **Base64 format**, which enables the graph to be displayed directly on the webpage without creating an extra image file.

Another important feature of our project is the **automatic PDF report generation**.

Using the **ReportLab library**, the application creates a professional report containing the selected numerical method, equation, interval, tolerance, and maximum iterations.

Finally, this report is sent directly to the user for download using Flask's **send_file()** function.

In summary, our implementation combines Flask for web development, SymPy for mathematical computation, Matplotlib for graphical visualization, JSON for data transfer, and ReportLab for PDF generation.

By integrating all these technologies, we successfully developed a complete web-based numerical analysis system that is interactive, user-friendly, and educational.

Now, I would like to invite our final presenter, who will demonstrate the complete website and explain its features and final output. Thank you.



Presentation Tip (Very Important)

When you explain this part, **don't just stand and read the script**. Open your **app.py** in VS Code and point to the code as you speak. For example:

- When you say "**We created a function called `parse_equation()`**", highlight that function.
- When you mention "**The Home Route**", point to `@app.route('/')`.
- When explaining the **POST request**, point to `if request.method == 'POST':`.
- When talking about **validation**, highlight `if f(a) * f(b) >= 0:`.
- When discussing the **Bisection Method**, point to `c = (a + b) / 2`.
- When discussing the **Regula Falsi Method**, point to `c = b - (fb * (a - b)) / (fa - fb)`.
- When explaining the **graph**, highlight the **Matplotlib** code.

- When explaining the **PDF**, point to the `download_pdf()` function.

Doing this will make it clear to your lecturer that you understand **how your own implementation works**, not just the underlying theory.

.

Member 5 Presentation Script (Website Demonstration + Conclusion)

(20–25 lines | Around 3–4 minutes)

Thank you. Finally, I will demonstrate our web application and conclude our presentation.

When the application starts, the user first sees the **Home Page**, where both numerical methods are displayed with a brief description.

The user can choose either the **Bisection Method** or the **False Position Method** by clicking the **Open Calculator** button.

After selecting a method, a calculator page opens where the user enters the mathematical equation, lower bound, upper bound, tolerance, and maximum number of iterations.

To make the application easier to use, we have also included **Quick Example Presets**, allowing users to test the application with a single click.

After entering the required values, the user clicks the **Calculate Root** button.

The application then executes the selected numerical method and calculates the approximate root.

The calculated root is displayed at the top of the result section.

Below the result, the application provides a complete **iteration table**, showing the values of **a**, **b**, **c**, and **f(c)** for every iteration.

Users can also click the **Toggle Plot Graph** button to visualize the function graph along with the calculated root.

Another important feature is the **Step-by-Step Animation**, where users can observe how the interval changes during each iteration using the **Previous**, **Next**, and **Auto Play** buttons.

Finally, users can click the **Export PDF Report** button to download a summary of the calculation for future reference.

Overall, our project combines numerical methods with modern web technologies to create an interactive and user-friendly learning platform.

This application not only calculates the root accurately but also helps users understand the complete solving process through tables, graphs, animations, and reports.

In the future, we plan to add more numerical methods such as the **Newton-Raphson Method**, **Secant Method**, and **Fixed Point Iteration**, making the application even more useful for students.

We believe this project demonstrates how Python and Flask can be used to simplify complex mathematical concepts through an interactive web application.

This concludes our presentation. Thank you for your valuable time and attention. We will now be happy to answer any questions.